

Security Posture of This Repository

Operation fsoc: Case Study

Gurmann Basran

Revised 2026-06-15 (rev. 3).

1 What this repository is, and is not

This repository is a case study, not a runbook. Its purpose is to document the engineering process behind a security-focused infrastructure project in enough detail to be useful to another engineer, a hiring manager, or an evaluation reviewer looking at my work, while publishing as little as possible about the underlying system itself.

A case study answers “how did you think about this problem?” A runbook answers “what exactly did you configure?” I am publishing the first one. The second one lives in a private planning directory alongside the live system it describes, and it is going to stay there.

The redaction philosophy I have applied across all seven documents is simple. Everything that could plausibly help an attacker fingerprint, target, or exploit my specific deployment has been kept out. Everything that communicates the reasoning, the methodology, and the design principles has been kept in.

2 What is deliberately out

- Specific product names, for any tool, at any layer.
- Category-level descriptions narrow enough that the product behind the category is inferable. No “crowd-sourced behavioral defense,” no “modern UDP VPN,” no “recursive DNS resolver with a chained filter,” no “open-source hypervisor with storage replication” phrasing. The writeup stays above the layer where a reader familiar with the field could name the product.
- The operator workstation’s software stack, named at any level: the distribution, the desktop and window-management layer, the terminal and shell environment, the editor, the file manager, the bootloader, and the on-disk encryption and snapshot layout. The workstation migration is described in document 7 by its design reasoning and its threat-model effect, never by its parts list. The combination is a fingerprint of the most-privileged endpoint in the system, which is exactly the machine I least want to describe.
- Specific version numbers, for any tool, anywhere.
- Specific counts of inbound ports, outbound ports, or per-port service identity.
- Specific architectural detail beyond “trust zones exist and have policies between them.” Exact subnets, VLAN tags, inter-zone rule counts, zone-to-zone permission matrices, DNS path internals, and service placement are not in the writeup.
- Real IP addresses, hostnames, MAC addresses, or domain names.

- Real firewall rule IDs, API user names, SSH key identifiers, certificate subject names, or credential-file paths.
- Contents of the operational `vault/` directory: keys, TLS private keys, backup-encryption recipients, provisioning secrets.
- Specific notification endpoints, upstream DNS provider names, upstream VPN provider names, domain registrar, ISP name, or certificate issuer.
- The geographic location of the hardware beyond country level.
- Script snippets, heuristic thresholds, detection timing, script-to-layer mapping, or the protocol-specific primitives each script reads. Scripts are described at the goal level only.
- Details of the phases that involve offensive capability (the physical-security phase, the advanced-track phase) beyond a generic mention that they exist.
- A catalogue of any physical-security tooling kit: hardware model names, firmware versions, serial numbers.
- The specific thresholds, intervals, and capacity numbers the operational-health watchdog triggers on. The method (watch a trend rather than a snapshot, alert before the crash rather than after) is in this writeup; the operator-tuned numbers are not, because they describe the exact headroom of specific hosts.
- The physical escrow arrangement for the backup encryption key, beyond the fact that an off-fleet copy exists and has been verified current. Where it is kept, and how, is out.
- The internals of the orchestration and coordination layer the project now runs on: the specific tools, the governance thresholds, and the change-class table that decides what runs unattended. The design reasoning is in the Overview and Scripts documents; the mechanism is not.
- The destination of the planned hardware relocation, beyond the fact that it is a move to a new site behind a new upstream provider. It is described only as a forcing function for recovery, never by where it is going.

3 What is deliberately in

- The motivation for the project and the identity of the person running it.
- The threat-model structure, including the assume-breach scenario table with blast-radius limits and detection signals at the category level, and the definitions I use for severity categories.
- The finding-with-traceability audit methodology, without the contents of any specific findings.
- Script design principles and the common patterns across every script in the set, at the goal level.

- Integration traps at the “class of trap” level, without the specific tool names that would help someone fingerprint my stack.
- The reasoning behind the workstation migration and the residual risk it does and does not close, at the design level, with the parts list left out.
- The operational-health and capacity methodology: that it is a distinct audit dimension, that it is checked continuously rather than at a point in time, and the reasoning for why a security audit alone does not cover it.
- The recovery-verification methodology: proving the backups actually restore rather than assuming they do, escrowing the recovery key off the fleet and verifying that copy is current, and alarming on backup staleness.
- My professional identity: name, field of study, country, the fact that this is a personal project of mine.

The test I apply when deciding whether something goes in the public repo or stays in the private planning directory is: “if this fell into the hands of an attacker, would it help them more than it has already helped me to publish it?” The methodology and the reasoning pass that test. The specific tool names and configuration do not.

4 Responsible disclosure

If you believe any of my redactions is insufficient, meaning you have found something in this repository that meaningfully exposes the underlying deployment, please open a GitHub issue on the repo marked `security`. I take it seriously.

In particular, I am interested in being told about:

- Any real IP, hostname, or domain I missed during sanitization.
- Any product name, version number, or configuration fragment that can be reversed to identify specific versions or patch states of my live deployment.
- Any text that unintentionally confirms which of several plausible tools I use for a given layer.
- Any inference chain I may have underestimated: patterns across the documents that combine to reveal more than any single document does on its own.
- Any other detail whose presence I should not have allowed through the sanitization pass.

This repository is MIT-licensed, so reuse is welcome. The redacted content is not something I expect to publish in the future; the live deployment is a private security posture, not a demo.